⚡ AdaBoost — Step by Step

1) Core Idea (one line)

Train many **very simple "weak" learners** one after another; at each round, **focus more on the points previously misclassified**. Combine them with **weighted votes** to get a strong model.

---

2) Setup & Notation

- Binary labels: $y_i \in \{-1, +1\}$, features x_i, $i = 1, \ldots, N$.
- Weak learner at round t: $h_t(x) \in \{-1, +1\}$ (often a **decision stump** = tree of depth 1).
- Sample weights at round t: $w_i^{(t)}$, initialized uniformly $w_i^{(1)} = \frac{1}{N}$.
- Number of rounds (estimators): T.

---

3) The Training Loop (AdaBoost.M1, binary classification)

Repeat for $t = 1, \ldots, T$:

(a) Fit weak learner on weighted data

Train h_t to minimize **weighted error**:

$$\varepsilon_t = \sum_{i=1}^{N} w_i^{(t)} \cdot \mathbf{1}[y_i \neq h_t(x_i)] .$$

(b) Compute learner weight (its "vote")

$$\alpha_t = \tfrac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right) .$$

- Better learner (smaller ε_t) ⇒ larger α_t ⇒ stronger vote.
- If $\varepsilon_t \geq 0.5$, the learner is too weak—**stop or retry** (it's no better than chance).

(c) Update sample weights

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp\big(-\alpha_t\, y_i\, h_t(x_i)\big)}{Z_t},$$

where Z_t normalizes weights to sum to 1.

- If **misclassified**: $y_i h_t(x_i) = -1$ ⇒ weight × $e^{+\alpha_t}$ (goes **up**).
- If **correct**: $y_i h_t(x_i) = +1$ ⇒ weight × $e^{-\alpha_t}$ (goes **down**).

Final classifier (after T rounds)

$$F(x) = \sum_{t=1}^{T} \alpha_t\, h_t(x), \qquad \hat{y}(x) = \operatorname{sign}\big(F(x)\big).$$

(Think of $F(x)$ as a **score**; larger magnitude = more confidence.)

---

4) What AdaBoost is Minimizing (intuition)

AdaBoost performs stage-wise additive modeling to minimize **exponential loss**:

$$\mathcal{L} = \sum_{i=1}^{N} \exp\big(-y_i F(x_i)\big), \quad \text{where} \quad F(x) = \sum_t \alpha_t h_t(x).$$

This pushes the **margin** $m_i = y_i F(x_i)$ to be large and positive → **better generalization**.

---

5) Geometric Intuition (picture in your head)

- Round 1: a stump draws one coarse split → some errors remain.
- Reweighting **pulls** the next stump toward the hard region.
- Each stump adds a tiny **tilt**; summing many **straight cuts** forms a **flexible, curved boundary**.
- Persistent hard points keep tugging the boundary—unless they're noise/outliers.

---

6) Tiny Numerical Walkthrough (mental calculator friendly)

Suppose round t gave weighted error $\varepsilon_t = 0.25$.

- $\alpha_t = \frac{1}{2}\ln\frac{1-0.25}{0.25} = \frac{1}{2}\ln(3) \approx 0.5493$.
- Misclassified points' weights × $e^{+\alpha_t} \approx 1.732$ (become heavier).
- Correctly classified points' weights × $e^{-\alpha_t} \approx 0.577$ (become lighter).
- Normalize so weights sum to 1.

  Effect: Next stump pays **more attention** to previously misclassified points.

---

7) Multiclass AdaBoost (quick map)

- **SAMME** (discrete): extends the same idea; learner weight includes ln(K−1) where K = classes. Voting is by predicted class.
- **SAMME.R** (real): uses **class probabilities** from weak learners (usually better and faster to converge).

---

8) AdaBoost for Regression (high level)

- Common variant: **AdaBoost.R2**.
- Idea: compute a **normalized absolute loss** per sample; derive a round error err_t; set $\beta_t = \frac{\mathrm{err}_t}{1-\mathrm{err}_t}$.
- Increase weights more for samples with **larger errors**.
- Combine weak regressors with weights related to $\log(1/\beta_t)$ (often via weighted median/mean).
- Same spirit: **focus on hard points**, but with continuous targets.

---

9) Practical Hyperparameters (what to tune first)

- **n_estimators (T):** 50–300 to start. More rounds = finer boundary (watch overfitting on noisy data).
- **learning_rate (shrinkage):** scales αt\alpha_t. Smaller (0.05–0.5) = steadier learning; may need more estimators.
- **base_estimator:** decision stumps (trees with max_depth=1) are standard; shallow trees (depth 2–3) can add flexibility.
- **algorithm:** for multiclass, prefer **SAMME.R** when probabilities are available.

---

10) Strengths & Caveats

Why it works well

- Sequential focus on hard cases → **bias ↓**.
- Simple base learners + weighted voting → often **variance not too high**.
- **Margin maximization** → strong generalization in practice.

Watch out for

- **Label noise/outliers:** they get large weights and can hijack training.
  Mitigate with smaller learning_rate, early stopping, capped tree depth, robust preprocessing.
- **Class imbalance:** use initial class-balanced weights or cost-sensitive metrics.
- **Error ≥ 0.5:** weak learner is too poor; stop or adjust base model.

---

11) When to Use AdaBoost

- Strong choice with **tabular data** and **shallow trees** when you need a **fast, accurate**, and **interpretable-ish** ensemble (feature importances from trees still help).
- Prefer **Gradient Boosting / XGBoost** when you need more **regularization knobs** or performance on **very large/complex** datasets.
- Prefer **Random Forest** when data is **noisy** and you want **maximum robustness** (variance reduction without sequential reweighting).

---

12) Real-World Pattern (how you'll apply it)

1. Start with decision stumps, n_estimators=200, learning_rate=0.1.
2. Check validation AUC/accuracy (classification) or RMSE/MAE (regression).
3. If **underfitting** → increase n_estimators or allow slightly deeper trees.
4. If **overfitting / noisy** → decrease learning_rate, add early stopping, or clean labels/outliers.
5. Inspect feature importances & misclassified regions to guide feature engineering.

Memory Hook

AdaBoost = *Add* weak learners **sequentially**, **boost** focus on mistakes, and **vote** by confidence. Each round bends the boundary toward the tough spots.
That's the whole story—concept, math, and practice—so it sticks whenever you come back.